Graph-based learning

In this final chapter, we will demonstrate how ideas and tools from graph theory can be leveraged to build powerful models, and how graph-based learning is on the rise in a wide range of current research applications. We will see that methods inspired by graph theory are both efficient and effective for data analysis. Furthermore, many aspects will be closely related to concepts described in the chapters above, and have served as inspirations for extensions of algorithms that were originally formulated from different perspectives.

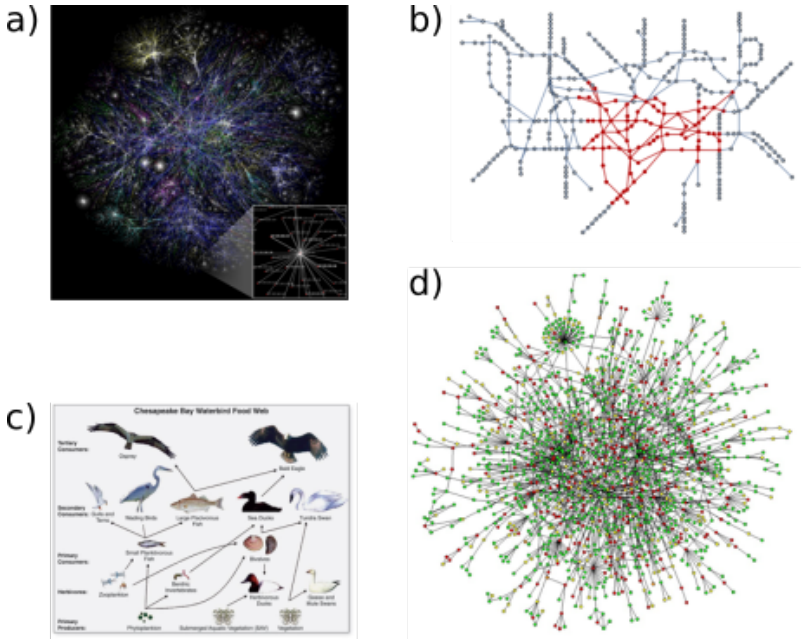


Figure 11.1. A few real-world examples of graphs. a) An illustration of the internet, where each node is a server and the edges are coloured according to traffic between them. b) A schematic of an Underground Map (unfortunately, the real map for the London underground is under heavy copyright rules and cannot be used here). The tube network is a graph of stations connected by the lines between them. c) Food webs are also a type graph, with each interaction being a predator-prey relation between species of animals. d) The interactions between proteins in a cell or a bacterium (in this case *Saccharomyces cerevisiae* or more commonly known as yeast) form a graph too.

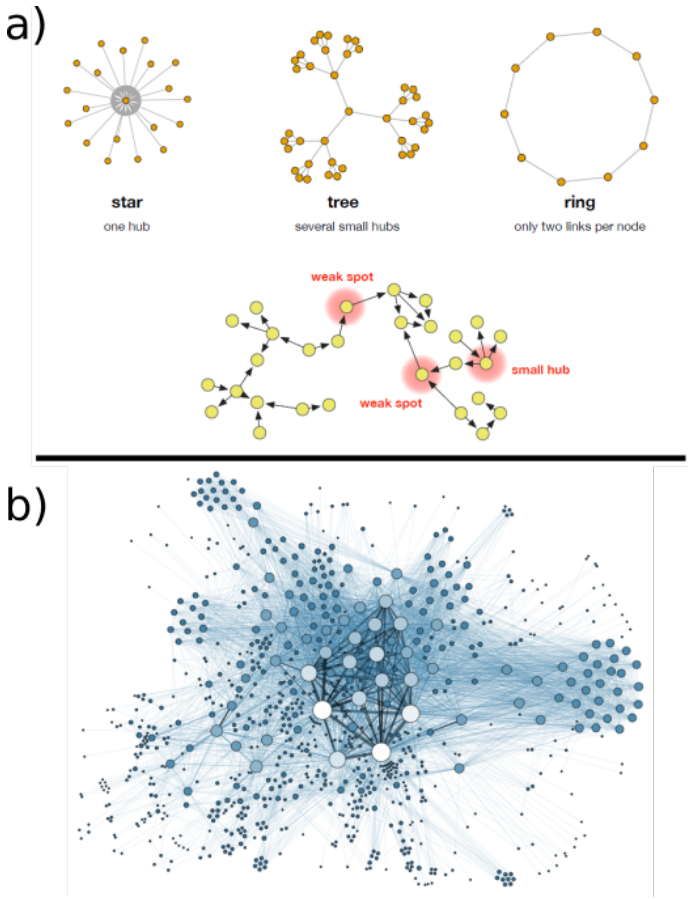


Figure 11.2. a) Small networks are intuitive, b) but large ones are not (at all). This motivates the use of graph-based learning methods to understand highly complex data.

1. Graph theory preliminaries

1.1. Graphs and data. To link the use of graphs for data analysis with our previous chapters, we start once more with a data set, with N samples, written in the usual format as a vector in p-dimensional space:

$$\{\mathbf{x}^{(i)}\}_{i=1}^{N} \qquad \mathbf{x}^{(i)} \in \mathbb{R}^p \qquad \mathbf{x}^{(i)} = \begin{pmatrix} x_1^{(i)} \\ \vdots \\ x_p^{(i)} \end{pmatrix}$$

As before, this can be summarised in matrix form:

$$X_{N\times p} = \begin{pmatrix} {\mathbf{x}^{(1)}}^T \\ \vdots \\ {\mathbf{x}^{(N)}}^T \end{pmatrix}$$

In the previous chapters, the goal was to find separable subsets, which was coupled with some notion of distance. Likewise, with the dimensionality reduction methods, we were aiming to obtain projections in lower dimensions while capturing most of the variation in the data. In both cases, the geometry of the dataset played the most important role.

The key ingredient to many of these methods was the notion of similarity or dissimilarity (in some cases a distance) between samples. This yielded an $N \times N$ matrix summarising the

pairwise relationships. Examples of both similarities (S) and dissimilarities (D) included:

$$\begin{aligned}
(\textit{cosine similarity}) \qquad & S_{ij} = \frac{\mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)}}{\|\mathbf{x}^{(i)}\| \, \|\mathbf{x}^{(j)}\|} \\
(\textit{statistical similarity}) \qquad & S_{ij} = \frac{\mathrm{cov}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})}{\sqrt{\mathrm{cov}(\mathbf{x}^{(i)}, \mathbf{x}^{(i)})}\sqrt{\mathrm{cov}(\mathbf{x}^{(j)}, \mathbf{x}^{(j)})}} \\
(\textit{Euclidean distance}) \qquad & D_{ij} = \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\| \\
(\textit{radial kernel matrix }) \qquad & D_{ij} = \exp\left[-\frac{\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2}{c}\right]
\end{aligned}$$

In most of the unsupervised learning methods, we had to define pairwise (dis)similarities between samples, which were the important determinants of the structure of the dataset. In other words, what mattered most about our data sets where the pairwise interactions between them. This is the insight that leads into graph representations of data.

Even further, in many applications, the problem is in fact directly and only defined by the pairwise (dis)similarities, that is we do not know anything about the $\mathbf{x}^{(i)}$, but rather simply have a $N \times N$ matrix S or D, compiling the binary relationships between samples.

This is the basis for the definition of graphs in data analysis: **we will consider graphs with N nodes, where each sample is a *node* and the relationships between pairs of nodes (as given by the matrix S or D) will define E *(weighted) edges* between the nodes.** We now see a few definitions on graphs. (Many of you will have seen many of these concepts in the course in Network Science.)

> *Quick Aside:* Alternatively, in many applications, a $p \times p$ matrix of pairwise measures of statistical dependency of the features of the dataset can be estimated instead. This yields a network of interdependencies between the features of the dataset. These measures include statistical relationships such as *Pearson correlations*, *Mutual Information*, *Granger Causality*, *Cosine Similarity* etc. The networks obtained from these measures are useful in datasets where features represent specific parts of a system that the data represent. For instance, time-series data recorded from different parts of the brain can be transformed into a network of interactions among the brain regions to study the interaction structure of the brain. For a review of network construction and analysis used in neuroscience, see *Bassett, D., Sporns, O. Network neuroscience., Nat Neurosci 20, 353–364 (2017).*

1.2. Graphs: some definitions. A *graph* is defined to be a combinatorial object on two sets: a set of nodes $V = \{i\}_{i=1}^N$ (aka vertices) and a set of edges $E = \{(i,j)\}_{i \sim j}$ (aka links). In mathematical terms, a graph is then defined as $G(V, E)$.

In the context of graph-based learning, the parallels are now obvious: the set of nodes is the set of samples:

$$\{\mathbf{x}^{(i)}\}_{i=1}^N \mapsto \{i\}_{i=1}^N$$

and the edges are defined from the entries of S or D.

In Figure 11.3, we show a very simple and basic example of a graph where $N = 4$. In this specific example, we have the following node and edge sets:

$$\begin{aligned}
V &= \{1, 2, 3, 4\} \\
E &= \{(1,2), (2,3), (2,4)\}
\end{aligned}$$

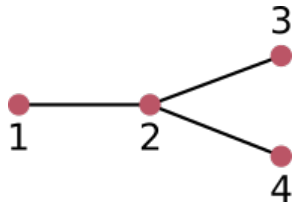


Figure 11.3. A simple example of a graph with $N = 4$.

This can be easily and equivalently summarised using the *adjacency matrix* $A_{N\times N}$. In the example above, this is:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

The definition of the adjacency matrix is therefore:

$$A_{ij} = \begin{cases} 1, & \text{if nodes } i \text{ and } j \text{ are connected, i.e. } (i,j) \in E \\ 0, & \text{otherwise} \end{cases}$$

From the adjacency matrix, we can also obtain the *degree vector*:

$$\mathbf{d} = A\mathbf{1} = \begin{bmatrix} 1 \\ 3 \\ 1 \\ 1 \end{bmatrix}$$

As you can see, the *degree* of a particular node is simply the number of edges it makes with other nodes. In our example above, node 1 has degree 1, node 2 has degree 3, and so on.

Note that the graph above is *undirected*, i.e., there is no direction associated to the edges, which implies that $A = A^T$.

1.3. Directed graphs. For the case of a *directed graph*, the latter equality does not necessarily hold and thus in most cases in fact we have $A \neq A^T$. An example of a directed graph is given in Figure 11.4.

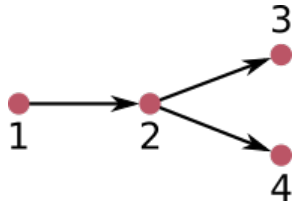


Figure 11.4. A simple example of a *directed* graph.

The corresponding adjacency matrix is given by:

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Note that for directed graphs, there are two types of degrees: the *in-degree* (the number of incoming edges of a particular node) as well as the *out-degree*, which can be written from the adjacency matrix as:

$$\mathbf{d}_{\text{in}} = A\mathbf{1} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\mathbf{d}_{\text{out}} = A^T\mathbf{1} = \begin{bmatrix} 1 \\ 2 \\ 0 \\ 0 \end{bmatrix}$$

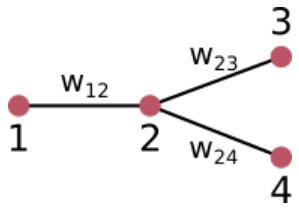


Figure 11.5. An example of a weighted graph.

1.4. Weighted graphs. These are graphs where each edge has an associated *weight*, in most cases a number in $\mathbb{R}$. We then denote the weight between the nodes i and j by w_{ij}. In the example of Figure 11.5, the adjacency matrix would then look like the following:

$$A = \begin{bmatrix} 0 & w_{12} & 0 & 0 \\ w_{12} & 0 & w_{23} & w_{24} \\ 0 & w_{23} & 0 & 0 \\ 0 & w_{24} & 0 & 0 \end{bmatrix}$$

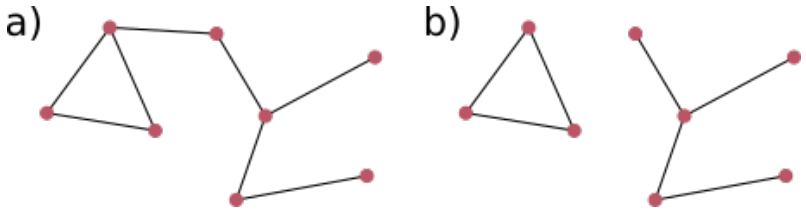


Figure 11.6. a) An example of a connected graph. **b)** An example of a disconnected graph with two components.

1.5. Connected graphs. Finally, we also define the concept of *connected graph*, as a graph where every possible pair of nodes is connected by a path. Figure 11.6 gives examples of both a connected as well as a disconnected graphs.

A disconnected graph has multiple components, whereas a connected graph has only one component. It can be shown trivially that if the graph is connected, the adjacency A has full rank and cannot be rearranged in block-diagonal form (i.e., the matrix is irreducible). You can immediately see that the clustering problem on graphs is directly related to making the adjacency matrix as block-diagonal as possible by relabelling the nodes (as seen in Chapter 9). Remembering that the nodes are samples, a good clustering implies

that the graph is 'close' to being disconnected, i.e., it could be made disconnected by cutting a few edges (the *graph partitioning* problem in graph theory).

Note that for directed graphs there exist two types of connectivity, strong and weak. In the weaker case, the definition is the same as for undirected graphs: If all directed edges were to be replaced by regular edges, one would get a connected graph. However, being strongly connected means that there is a *directed* path for every pair of nodes.

1.6. Distances on graphs. Another important concept in graph-based learning is that of *graph distance*, to be understood as *distance on the graph*. As with the concept of (dis)similarities, there are many different definitions that can be used. Two of the most popular measures of graph-based distances use the concept of paths on the graph:

- The **geodesic distance** is perhaps the most popular and is simply defined to be the *minimal* number of edges between two nodes, i.e., if we denote the set of all paths between i and j as $\{i, j\}$, we can write the geodesic distance as:

$$\min_{\{i,j\}}(\#\text{ edges in }\{i,j\})$$

- The **average distance** between two nodes i and j is also simply the average length of all paths connecting them:

$$\langle \#\text{ edges}\rangle_{\{i,j\}}$$

Note that if the graph is weighted, then the length of a path is the sum of the weights of the edges on the path. This allows us to take into account some of the geometric information of the data set.

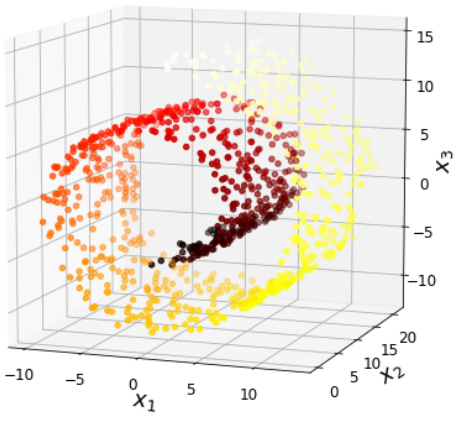


Figure 11.7. The 'Swiss roll' data set.

As we will see later, **distances on graphs can sometimes provide a much better representation of the intrinsic geometry of the data than Euclidean distances in** $\mathbb{R}^p$. One such dataset is the so-called *swiss roll* dataset, which is plotted in Figure 11.7. You can immediately see that a distance compiled on a well chosen graph representing the data set will be much more informative of the structure of this data set than the set of Euclidean distances in $\mathbb{R}^3$.

There are other measures of distance and of connectivity that emerge from considering dynamical processes on graphs related to diffusion. To introduce those, we now define the Laplacian matrix of a graph.

1.7. The Laplacian matrix of a graph. This matrix is essentially an extension of the Laplacian operator to graphs. It stems from the heat equation (aka the diffusion equation):

$$u = u(x,t), \quad \frac{\partial u}{\partial t} = \nabla^2 u$$

Here, ∇^2 is the Laplacian operator, defined as:

$$\nabla^2 u = \frac{d}{dx}\left(\frac{d}{dx}u\right)$$

If we apply a discretisation to space, i.e., $\mathbf{u} = \begin{pmatrix} u_1 & \dots & u_N \end{pmatrix}^T$, then we can write:

$$\begin{aligned} \nabla^2 u &= \frac{d}{dx}\left(\frac{d}{dx}u\right) \\ &\approx \Delta(\Delta u) = \Delta(u_{k+1} - u_k) \\ &= (u_{k+2} - u_{k+1}) - (u_{k+1} - u_k) \\ &= u_{k+2} - 2u_{k+1} + u_k \end{aligned}$$

In terms of graphs, the discretisation of the space can essentially be expressed as a graph of nodes along a line as shown in Figure 11.8.

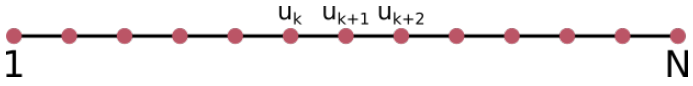


Figure 11.8. A discretisation of 1D space resulting in a graph consisting of a string of nodes connected to the next node by one edge each.

The heat equation on the discretised space can now be rewritten in terms of a tri-diagonal $N \times N$ matrix:

$$\frac{d\mathbf{u}}{dt} = \underbrace{\begin{pmatrix} -1 & 1 & & & & & \\ 1 & -2 & 1 & & & 0 & \\ & \ddots & \ddots & \ddots & & & \\ & & 1 & -2 & 1 & & \\ & & & \ddots & \ddots & \ddots & \\ & 0 & & & 1 & -2 & 1 \\ & & & & & 1 & -1 \end{pmatrix}}_{\text{Laplacian matrix: } -L} \begin{pmatrix} u_1 \\ \vdots \\ u_k \\ u_{k+1} \\ u_{k+2} \\ \vdots \\ u_N \end{pmatrix} \tag{11.1}$$

Note that the adjacency matrix of the graph in Figure 11.8 is given by:

$$A = \begin{pmatrix} 0 & 1 & & \\ 1 & 0 & \ddots & \\ & \ddots & \ddots & 1 \\ & & 1 & 0 \end{pmatrix}$$

and the degree matrix is $\mathcal{D} = \text{diag}(\mathbf{d}) = \text{diag}(A\mathbf{1})$.

It is then easy to see that the matrix in (11.1) is simply:

$$L = \mathcal{D} - A$$

L is called the *Laplacian matrix*. More specifically, L is commonly termed the *Combinatorial Graph Laplacian*. The spectral decomposition of the Laplacian matrix plays an important role in clustering and dimensionality reduction in graph-based learning, which we will explore in the following sections.

2. Clustering on graphs – graph partitioning

As mentioned above, there are two main problems that frequently appear in the field of unsupervised learning: dimensionality reduction and clustering. Graph-based learning methods are particularly suited for the latter but can also be used to do the former. We will first introduce approaches of how to tackle the problem of clustering data through graph-based methods. Then in Section 3 we will see how graphs allow us to perform dimensionality reduction – in particular, *nonlinear* dimensionality reduction.

As discussed in Chapter 9, clustered data should be similar within clusters and dissimilar between clusters. In graphs, this is analogue to the idea of *graph partitioning*, i.e. finding splits of the graph into subgraphs so that the nodes are partitioned into mutually exclusive subsets. Equivalently, one is trying to find cuts across a set of edges to produce partitions. If cutting a graph into separate subgraphs is 'easy', this is indicative of the presence of potentially good clusters.

2.1. Graph construction. In the introduction to this chapter, we mentioned that in some cases data is given directly in the form of a graph (e.g., a social network, or a citation graph), whereas in other cases our data might be a collection of samples in the form of high-dimensional vectors $\mathbf{x}^{(i)} \in \mathbb{R}^p, \quad i = 1, \dots, N$. In this second case, it might still be advantageous to describe the data set as a graph so that, e.g., one can approach the clustering problem as a graph partitioning problem.

Naturally, the first step to doing clustering using graphs is the graph construction from data. If we have traditional tabular data, graph construction usually starts from matrices of (dis)similarities $D_{N\times N}$ or $S_{N\times N}$, as described in Section 1.1. As an alternative viewpoint to multivariate statistics/calculus, these matrices can then be viewed as *adjacency matrices* of weighted complete graphs, i.e. a graph where every node is connected to every single other node via a weighted edge such that the adjacency matrix is 'full' (the graph in these cases is also called 'fully connected'). However, from a graph-theoretical perspective such 'weighted complete graphs' are usually not very helpful—they are too dense, with little structure and are not suited for the analysis using graph properties. Hence they tend not to be used in this form, and a *sparsification* method is usually applied to reduce the number of edges, i.e., to create zeros in the adjacency matrix. There are a few methods that are applied widely (but this is a very active area of current mathematical research):

- **Thresholding** is simply removing all edges below (or above) a certain threshold. This is usually not the best strategy, but it is done sometimes in very simple applications. Things that can go wrong are graphs that are sensitive to tiny changes in the threshold, where parts can become disconnected as a result.
- **Geometric graphs:**
  - **ε-ball:** This method only connects nodes that are within some distance ε of each other, where ε is a hyperparameter to be tuned.
  - **k-NN:** This is similar to the above, except the k nearest neighbours of each node will be connected to it. Here k is a hyperparameter to be tuned. There

is a wide set of variations on k-NN constructions: standard k-NN, mutual k-NN, and continuous k-NN. [1]

Quick Aside: sparsifications based on the minimum spanning tree. The minimum spanning tree (MST) of the graph connects all N nodes of the graph with just $N-1$ edges such that the sum of the weights of the chosen edges is minimal. Once the MST is found (there are good algorithms for this), then edges are added according to different criteria (spectral or distance based) up to a given sparsity regulated by a given hyperparameter. The spectral sparsification algorithm by Spielman and Srivastava[a] is a beautiful example of this approach.

[a] *Graph Sparsification by Effective Resistances*, Daniel A. Spielman and Nikhil Srivastava, SIAM Journal on Computing 2011 40:6, 1913-1926

It should be evident from our descriptions above that the level of sparsity can be adjusted with one (or more) parameters (e.g., k in k-NN or ε in ε-ball). These can be thought in many cases as hyperparameters to be chosen with a particular task in mind so that the properties of the data set are best captured by the graph.

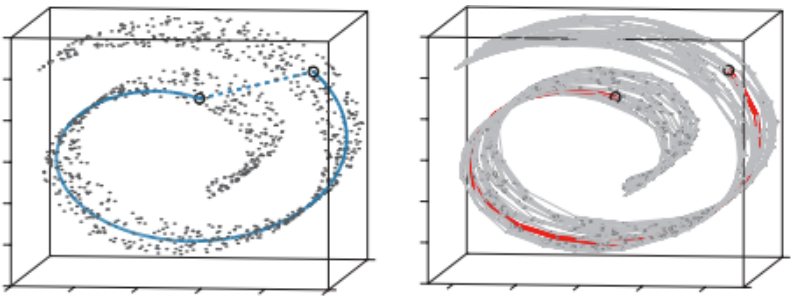


Figure 11.9. The Swiss roll data set with a geometric graph. This example gives a visual intuition of why a geometric graph (say ε-ball or k-NN) with a well chosen sparsity could be a better representation of this data set (as compared to a full distance matrix $D_{N\times N}$ in Euclidean space). Why random walks or diffusions on such a graph could be used to capture the geometry of the data set is also immediately evident from the picture.

2.2. Spectral partitioning of graphs. Once we have a graph, we can apply some of the results we have obtained during this course and in this chapter to compute a partitioning of the graph into separate subgraphs. Similarly to methods described previously in this course, this process is usually linked to a cost function. In the case of graph partitioning, the cost function is oftentimes (but not always!) related to the 'size of the cut' (i.e., the number of edges or the sum of the weights of the edges) that are necessary to be cut so as to split the graph into k separate subgraphs. The separated subgraphs will correspond to the clusters of data points. Mathematically, we can formalise this notion as follows. We will consider in detail the problem of bipartitions, i.e., splitting a given graph into $k = 2$ clusters S_1 and S_2. (The ideas behind this approach can be extended to $k > 2$ but the extensions are not trivial and will be mentioned at the end of the section.)

Given S_1 and S_2, the cost function that counts the number of edges needed to split the graph into two subgraphs (i.e., the *cut*) can be written in terms of the graph adjacency matrix A as follows:

$$C = \frac{1}{2} \sum_{i\in S_1, j\in S_2} A_{ij}$$

[1] *Geometric graphs from data to aid classification tasks with graph convolutional networks*, Y. Qian, P. Expert, P. Panzarasa, M. Barahona, Patterns 2, 4 (2021): 100237.

Note how we are selecting the edges that link a node i in S_1 with a node j in S_2. In order to simplify our derivation, it is helpful to define an indicator variable s_i:

$$s_i = \begin{cases} +1 & \text{if } i \in S_1 \\ -1 & \text{if } i \in S_2 \end{cases}$$

from which we have:

$$t_{ij} = \frac{1}{2}(1 - s_i s_j) = \begin{cases} 1 & \text{if } i,j \text{ are in different clusters} \\ 0 & \text{if } i,j \text{ are in the same cluster} \end{cases}$$

With these definitions, we rewrite the cost function as:

$$C = \frac{1}{4}\sum_{i,j} t_{ij} A_{ij} = \frac{1}{4}\left(\sum_{i,j} A_{ij} - \sum_{i,j} A_{ij} s_i s_j\right)$$

Note that we can rewrite:

$$\sum_{i,j} A_{ij} = \sum_i \left(\sum_j A_{ij}\right) = \sum_i d_i = \sum_i d_i s_i^2 = \sum_{i,j} d_i s_i s_j \delta_{ij}$$

where δ_{ij} is the Kronecker delta. Then we finally have:

$$C = \frac{1}{4}\sum_{i,j} \big(\underbrace{d_i \delta_{ij} - A_{ij}}_{L_{ij}}\big) s_i s_j = \frac{1}{4}\mathbf{s}^T L \mathbf{s} \tag{11.2}$$

where we have rewritten the cost function in terms of L, the (combinatorial) graph Laplacian.

Our goal is thus to find a partition with a small cut. That is, we want to minimise the cost function:

$$\min_{\mathbf{s}\in\{-1,+1\}^N} \mathbf{s}^T L \mathbf{s} \qquad \text{under the constraints } \begin{cases} \mathbf{s}^T \cdot \mathbf{s} & = n_1 + n_2 = N \\ \mathbf{s}^T \cdot \mathbf{1} & = n_1 - n_2 \end{cases} \tag{11.3}$$

Note that $\mathbf{s} \in \{-1,+1\}^N$ by definition. Unfortunately, the optimisation 11.3 cannot be solved analytically as it is a combinatorial optimisation (the aim of the optimisation is to assign two discrete labels to the nodes of the graph). Hence this is a 'hard' optimisation problem and, as we know from earlier chapters, complete enumeration of the space of graph partitions is out of computational bounds for even small graphs.

To solve this problem in practice, we rely on optimisation techniques. As discussed from the beginning of the course, one such method is called *relaxation*, which aims to solve an easier problem that is closely related to the original one. In this case, the relaxation is to solve the optimisation (11.3) for $\mathbf{s} \in \mathbb{R}^N$ rather than in the original discrete space $\{-1,+1\}^N$ (a hypercube). Now that we have a continuous optimisation, we can use the method of Lagrange multipliers for constrained optimisation. We thus define the Lagrangian with two Lagrange multipliers λ, μ:

$$\mathcal{C}(\mathbf{s},\lambda,\mu) = \mathbf{s}^T L \mathbf{s} + \lambda(N - \mathbf{s}^T \cdot \mathbf{s}) + 2\mu\big((n_1 - n_2) - \mathbf{s}^T \cdot \mathbf{1}\big), \qquad \mathbf{s} \in \mathbb{R}^N, \lambda \in \mathbb{R}, \mu \in \mathbb{R}.$$

As we have done many times before, we now seek $\mathbf{s}^\star$ such that:

$$\nabla_{\mathbf{s}} \mathcal{C}\big|_{\mathbf{s}^\star} = 0.$$

At this $\mathbf{s}^\star$, we then have:

$$L\mathbf{s}^\star = \lambda \mathbf{s}^\star + \mu \mathbf{1} \tag{11.4}$$

Using $L\mathbf{1} = 0 = \mathbf{1}^T L$, we know that:

$$\mathbf{1}^T L\mathbf{s}^\star = \lambda \underbrace{\mathbf{1}^T \cdot \mathbf{s}^\star}_{=(n_1-n_2)} + \mu \underbrace{\mathbf{1}^T \cdot \mathbf{1}}_{=N} = 0$$

And hence:

$$\frac{\mu}{\lambda} = -\frac{n_1 - n_2}{N} \tag{11.5}$$

We then solve equation (11.4) for $\mathbf{s}^\star$:

$$L\mathbf{s}^\star = \lambda\left(\mathbf{s}^\star + \frac{\mu}{\lambda}\mathbf{1}\right)$$

Again, because $L\mathbf{1} = 0$, we can add a term to the right hand side so that:

$$L\mathbf{s}^\star = L\left(\mathbf{s}^\star + \frac{\mu}{\lambda}\mathbf{1}\right) = \lambda\left(\mathbf{s}^\star + \frac{\mu}{\lambda}\mathbf{1}\right)$$
$$\implies L\mathbf{v} = \lambda\mathbf{v}$$

where we have defined:

$$\mathbf{v} := \mathbf{s}^\star + \frac{\mu}{\lambda}\mathbf{1}. \tag{11.6}$$

This is the main result: the solution to this (relaxed) optimisation problem is given by **an eigenvector of the Laplacian matrix v and its associated eigenvalue** λ.

The remaining question is now: *which eigenvector is it?* If we plug this result into the cost function (11.2), we get:

$$C^\star = \frac{1}{4}\mathbf{s}^{\star T} L\mathbf{s}^\star = \frac{1}{4}\mathbf{v}^T L\mathbf{v} = \frac{1}{4}\lambda\,\mathbf{v}^T\mathbf{v}$$

where we have:

$$\begin{aligned}\mathbf{v}^T\mathbf{v} &= \mathbf{s}^{\star T}\mathbf{s}^\star + 2\frac{\mu}{\lambda}\mathbf{1}^T\mathbf{s}^\star + \frac{\mu^2}{\lambda^2}\mathbf{1}^T\mathbf{1} \\ &= N - 2\frac{(n_1-n_2)^2}{N} + \frac{(n_1-n_2)^2}{N} = \frac{(n_1+n_2)^2 - (n_1-n_2)^2}{N} = 4\frac{n_1 n_2}{N}.\end{aligned}$$

Thus, the cost function is

$$C^\star = \lambda\frac{n_1 n_2}{N}.$$

In order for $C^\star$ to be minimal, we will want λ to be the smallest possible eigenvalue (and $\mathbf{v}$ to be its associated eigenvector). However, note that $\lambda_1 = 0$ with $\mathbf{v}_1 = \mathbf{1}$ *cannot* be the solution we are looking for. To see this, use the definition (11.6) and multiply on the left by the vector $\mathbf{1}^T$:

$$\mathbf{1}^T\mathbf{v} = \mathbf{1}^T\mathbf{s}^\star + \frac{\mu}{\lambda}\mathbf{1}^T\mathbf{1} = (n_1 - n_2) - \frac{n_1 - n_2}{N}N = 0.$$

This shows that the solution must be orthogonal to $\mathbf{v}_1$.

Key message: The lowest value of the graph partitioning cost function $C^\star$ is achieved by choosing the second (i.e., the first non-trivial) eigenvector $\mathbf{v}_2$ with eigenvalue λ_2, which gives:

$$C^\star_{\min} = \lambda_2\frac{n_1 n_2}{N}$$

This is the classic result by Miroslav Fiedler, who obtained it in 1973[a].

[a] Algebraic connectivity of graphs, Miroslav Fiedler, Czechoslovak Mathematical Journal 23, 298-305 (1973)

The second eigenvector of the Laplacian, $\mathbf{v}_2$, is thus called the *Fiedler* eigenvector. Note that the quality of the bipartition is directly related to the eigenvalue λ_2, which is also called the *algebraic connectivity* of the graph. If $\lambda_2 \ll 1$ (is very small) then the cost of splitting the graph into two subgraphs is low, i.e., there exists a good bipartition of the graph given by $\mathbf{v}_2$. If, on the contrary, λ_2 is large (i.e., it is well separated from the zero eigenvalue $\lambda_1 = 0$) then the bipartition will not be 'good' since the cut cost function $C^\star$ cannot be made small.

Now you can see why λ_2 is called the algebraic connectivity of the graph. If the algebraic connectivity λ_2 is large, there is no good/easy bipartition of the graph because the cost of the bipartition is large; hence the graph is well connected. A small value of the algebraic connectivity means that a good bipartition with low cost exists; hence the graph is not well connected and it is easy to split into two subgraphs.

The Fiedler eigenvector is then used to produce the optimised bipartition according to its components. To see this, we need to 'undo' the relaxation from $\mathbb{R}^N$ back to a solution in $\{-1,+1\}^N$. Remember that the vector solution to the relaxation (in the reals) is:

$$\mathbf{s}^\star = \mathbf{v}_2 + \frac{n_1 - n_2}{N}\mathbf{1}, \quad \mathbf{s}^\star \in \mathbb{R}^N.$$

We would like to find a vector $\mathbf{s} \in \{-1,+1\}^N$ as close as possible to $\mathbf{s}^\star$ by maximising the projection of $\mathbf{s}$ on $\mathbf{s}^\star$:

$$\max_{\mathbf{s}\in\{-1,+1\}^N} \mathbf{s}^T \cdot \left(\mathbf{v}_2 + \frac{n_1 - n_2}{N}\mathbf{1}\right) = \max_{\mathbf{s}\in\{-1,+1\}^N} \mathbf{s}^T\mathbf{v}_2 + \frac{(n_1 - n_2)^2}{N}, \tag{11.7}$$

where we have used the fact that $\mathbf{1}^T\mathbf{s} = n_1 - n_2$.

There are a couple of procedures to maximise the alignment with the solution of the relaxation given by $\mathbf{v}_2$:

- If n_1 and n_2 are given, compute $\mathbf{v}_2 = (v_2^{(1)}, \ldots, v_2^{(N)})$ and order the elements $v_2^{(i)}$ from most positive to most negative. Assign the top n_1 elements to cluster 1 and the rest $n_2 = N - n_1$ to cluster 2.
- If we do not have a prescribed size for the split subgraphs, compute $\mathbf{v}_2 = (v_2^{(1)}, \ldots, v_2^{(N)})$ and assign all elements with $v_2^{(i)} < 0$ to cluster 1 and all elements $v_2^{(i)} > 0$ to cluster 2.

Both of this procedures aim to maximise the alignment (11.7) of the discrete solution.

2.2.1. ***Beyond bipartitions:*** The bipartition procedure described above can be generalised to arbitrary $k > 2$ in a number of ways. Here we briefly mention a couple with a very different flavour.

Sequential bipartitions: One approach is divisive ('top-down') by applying Fiedler bipartitions recursively. One would start from the full graph and iterate this process to generate a series of bipartitions, where each subgraph obtained as the result of a bipartition is again split until a particular end condition is met.

Using more Laplacian eigenvectors: Another approach is a generalisation that uses at once the full eigendecomposition of the Laplacian L. The standard algorithm is as follows. Given a diagonalisation of the Laplacian, i.e. $L = V\Lambda V^T$, we have the following steps:

(1) Take the $N \times m$ matrix $V_m = \begin{bmatrix}\mathbf{v}_1 & \ldots & \mathbf{v}_m\end{bmatrix}$ containing the ordered (by ascending eigenvalues) m eigenvectors.

(2) Each row (denoted $\mathbf{w}_i$) of this matrix, will give a description of each of the N nodes of the graph in terms of this m dimensional eigenspace. We use these as descriptors for each node.

(3) Carry out K-means on the vectors $\{\mathbf{w}_i\}_{i=1}^N$ to find K groups of nodes.

Quick Aside. Although our derivation has been based on the *combinatorial* Laplacian L, similar algorithms exist based on the *normalised* Laplacians $L_{\text{rw}}, L_{\text{sym}}$. These are defined below.

$$L_{\text{rw}} := I - A\mathcal{D}^{-1} = L\,\mathcal{D}^{-1}$$
$$L_{\text{sym}} := I - \mathcal{D}^{-\frac{1}{2}} A \mathcal{D}^{-\frac{1}{2}} = \mathcal{D}^{-\frac{1}{2}} L \mathcal{D}^{-\frac{1}{2}}$$

In such cases, the vectors $\mathbf{w}_i$ can either be obtained from L_{rw} — as in the **normalised cuts** algorithm of Shi and Malik; see *Normalized cuts and image segmentation,* IEEE Trans. Pattern Anal. Mach. Intell. 22(8), 888–905 (2000) — or from L_{sym} with an additional row-normalisation of the vectors $\mathbf{w}_i$ — as in the famous **spectral clustering** algorithm of Ng, Jordan and Weiss; see *On Spectral Clustering: Analysis and an algorithm*, Advances in Neural Information Processing Systems 14, 849–856 (2001). The normalised cuts algorithm using L_{rw} yields more balanced partitions and is therefore preferable. The spectral clustering algorithm using L_{sym}, on the other hand, does not extensively mention graphs, yet you can now see that some of the foundational ideas for data clustering are in fact intimately linked to ideas from graph partitioning. Between the two, we note that using L_{rw} is generally preferable over L_{sym}, since the eigenvectors of L_{rw} directly encode the cluster indicators whereas the eigenvectors of L_{sym} are additionally scaled by the square-root of node degrees which can result in undesirable behaviours — especially when the entries of the eigenvectors are small; for further details, see von Luxburg, *A tutorial on spectral clustering,* Statistics and Computing 17, 395–416 (2007) in 'additional reading'.

2.3. Modularity. As opposed to clustering methods based on spectral decomposition seen above, the *Modularity* method is somewhat based on combinatorial notions from graph theory. The idea is to induce a labelling of the nodes of a graph that lead to a maximally block-diagonal adjacency matrix. Interestingly, some of these notions can be related to dynamical aspects such as diffusions, conductance, and random walks on graphs.

In essence, Modularity is a cost function that has a similar starting point to those above, with some subtle differences. Of course, the overall aim is to find groups in the graph that are similar within and different to the rest. In other words, we want to find a partition of the graph that will have a maximally block-diagonal structure in the adjacency matrix, and has more 'blocks' than expected at random. The idea behind Modularity is to count the edges within blocks as well as between blocks. So this is very similar to our setup for clustering introduced in Chapter 9. To aid our explanations, let us consider the very

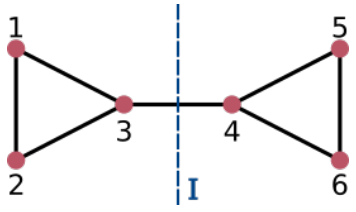


Figure 11.10. A simple example to illustrate Modularity. The blue dashed line is a good clustering split.

simple graph given in Figure 11.10 with adjacency matrix:

$$A = \begin{bmatrix} 0 & 1 & 1 & & & 0 \\ 1 & 0 & 1 & & & \\ 1 & 1 & 0 & 1 & & \\ & & 1 & 0 & 1 & 1 \\ & 0 & & 1 & 0 & 1 \\ & & & 1 & 1 & 0 \end{bmatrix}$$

Modularity is then a cost function for a given graph partition. In the simple graph in Figure 11.10, splitting the nodes into two clusters ($k = 2$) along the dashed blue line (denoted by the split number I) seems to be a good idea. Such a clustering can be succinctly written in matrix form through the *membership matrix* (see Chapter 9):

$$H_I = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}_{N \times k}$$

Here each row corresponds to a node in the graph (in this case nodes 1-6 as given in Figure 11.10) and each column represents one of the clusters (left, right).

Let us consider the following expression:

$$\frac{1}{2}(H_I^T A H_I)_{k \times k} = \begin{bmatrix} 3 & \frac{1}{2} \\ \frac{1}{2} & 3 \end{bmatrix}$$

This matrix captures the number of edges within each cluster (on the diagonal, 3 in each) as well as how many edges are between the clusters (on the off-diagonal, $\frac{1}{2}$ each). This information is in line with what we stated at the beginning: Modularity counts the number of edges within as well as between clusters, and attempts to find clusterings that will have most of the edges within the clusters and few edges between the clusters.

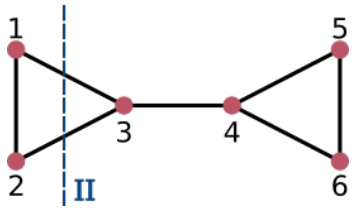


Figure 11.11. A simple example to illustrate Modularity. The blue dashed line is not a good clustering split.

In contrast, if we now consider a 'worse' split (denoted with the number II), shown by the blue dashed line in Figure 11.11, then we have a different membership matrix:

$$H_{II} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}_{N \times k}$$

and we compute again:

$$\frac{1}{2}(H_{II}^T A H_{II})_{k \times k} = \begin{bmatrix} 1 & 1 \\ 1 & 4 \end{bmatrix}$$

Again, this tells us that the first (left-hand side) cluster has only 1 edge, and the other cluster has 4. Since the split is through two edges this time, the inter-cluster counts amount to $2 \times \frac{1}{2} = 1$ for each cluster.

It is becoming apparent that a good partition will be achieved by maximising the total number of edges inside the defined clusters, i.e.,

$$\max_H \operatorname{Tr}\left[H^T A H\right].$$

Since the total number of edges E is fixed, maximising the number of edges inside clusters (diagonal elements of $H^T AH$) will reduce the number of edges between clusters (off diagonal elements of $H^T AH$).

The second part that makes Modularity so popular and successful is that it introduces a 'null model', against which the given partition is measured. This is usually called the *configuration model*, and is an essential ingredient to define an intrinsic measure of goodness of a given partition. The intuition behind the null model is to discount edges that are expected to be present at random given the properties of the graph. This can be formalised in the following way. We reduce the graph down to mere knowledge of the degrees of all the nodes, i.e. we only have the degree vector $\mathbf{d} = A\mathbf{1} = \begin{pmatrix} d_1 & \dots & d_N \end{pmatrix}^T$. Then we ask the question: If the nodes in the graph (with degree vector $\mathbf{d}$) were rewired at random, what would be the likelihood of the nodes being connected? The probability of two nodes i, j being connected is then given by:

$$R_{ij} = \frac{d_i d_j}{2E},$$

where E is the number of edges, i.e. $E = \frac{1}{2}\sum_{i=1}^{N} d_i$. In vector form, we can write this as:

$$R = \frac{1}{2E}\mathbf{d}\mathbf{d}^T$$

Now, the final step is to simply put the two concepts together to create the cost function which we call *modularity* :

$$Q = \frac{1}{2E}\operatorname{Tr}\left[H^T \underbrace{\left[A - \frac{1}{2E}\mathbf{d}\mathbf{d}^T\right]}_{Z} H\right]$$

The matrix Z is typically known as the Modularity matrix.

Key message: The aim of the Modularity method is simply to maximise the modularity Q over all membership matrices H:

$$\max_H Q. \tag{11.8}$$

In essence, we are looking for graph partitions with more in-cluster edges than you would expect at random. The graph clusters so found are usually called *graph communities* and modularity maximisation is usually referred as a *community detection* algorithm.

An important, distinctive feature is that the modularity Q is a compound cost function by definition since it has two balancing terms: one measuring the block-clustering, and another one that subtracts the expected clustering score of a null random model. Hence maximising modularity allows one to find an optimised number of clusters directly, without prescribing the number of clusters *a priori*, as we do in K-means.

As can be expected at this point, we recognise (11.8) as a combinatorial optimisation, since we need to optimise over the space of partitions. This is a 'hard' problem (in fact, it has been shown to be NP-hard) which needs to be solved through heuristic optimisations. There are many approaches to optimise modularity, from simulated annealing to specialised relaxations. However we mention two different approaches to the optimisation of Q in the space of partitions:

- Similar to spectral clustering, it can be shown that we can use the leading eigenvector of Z to maximise Q. Then we can effect bipartitions in a recurrent manner until Q does not increase. The stopping criterion is therefore $\Delta Q \leq 0$ in the iteration.
- In practice, modularity is optimised almost always through a greedy agglomerative algorithm that merges nodes of the graph into 'super-nodes' and proceeds sequentially until no improvement of Q is achieved. This method, which performs better than any other method currently known, is called the *Louvain algorithm*[2] (it was developed at UC Louvain in Belgium) and has become the industry standard and used very broadly in many areas of data science.

3. Dimensionality reduction using graphs

In this section we show how graphs enable us to perform dimensionality reduction in nonlinear settings. Specifically, we will explore how graph-based approximations can be used to learn nonlinear transformations of data that somehow capture the implicit geometry of data in the high-dimensional space. Here, we assume that data lie on some low-dimensional *manifold* of dimensionality m, embedded in a high-dimensional space of dimensionality $p > m$ – this is often called the *manifold assumption*. The notion of a manifold is central to the field of geometry in mathematics and is defined in terms of a collection of points such that locally the neighbourhood of each point resembles Euclidean space of dimensionality m, albeit the dimensionality of the "ambient space" in which the manifold lives can be $p > m$.

> *Quick Aside.* The dimensionality m of a m-manifold can be seen as the "number of linearly independent axes" if one were to zoom into the local neighbourhood of a point. Consequently, a 1-manifold is like a curve (say, a circle), a 2-manifold is like a surface (say, a disc), a 3-manifold is like a volume (say, a ball), and so on. To illustrate with some parametric examples: (a) the function $x^2 + y^2 = 1$ in $\mathbb{R}^2$ is a curve (the unit circle) in the XY-plane — a 1-manifold embedded in a 2-dimensional ambient Euclidean space — whereas in $\mathbb{R}^3$ it is a surface (a cylinder) in the XYZ-volume — a 2-manifold embedded in a 3-dimensional ambient Euclidean space, and (b) the parametrised curve $x(t) = \cos(t)$, $y(t) = \sin(t)$, and $z(t) = t$ is a curve (right-handed helix) in the XYZ-volume — a 1-manifold embedded in a 3-dimensional ambient Euclidean space.

The key idea is the following: **since graphs are also characterised by local neighbourhoods — a node connected to its neighbours via edges — constructing an "appropriate" graph can approximate the structure of the manifold on which data lie**. Consequently, we can use the techniques of graph construction from Section 2 to obtain a discrete approximation of the underlying manifold of the data.

[2] Blondel, V. D., Guillaume, J.-L., Lambiotte, R. & Lefebvre, E. Fast unfolding of communities in large networks. J. Stat. Mech. Theory Exp. 2008, P10008 (2008).

Figure 11.12 shows one such construction of a geometric graph using k-nearest neighbours with $k = 7$ on a Swiss roll dataset with $N = 1000$ data points. Once a graph has been (appropriately) constructed, we can now focus on the actual problem at hand — of mapping the data to its corresponding manifold which will be of a lower dimension. Similar to how PCA is motivated by the optimisation problem of minimising reconstruction error, **we can define optimisation problems that preserve desirable graph properties in the mapped space**, which is done using broadly two kinds of approaches.

3.1. Embeddings that preserve local structure. Since manifolds (and graphs) encode the local structure around each point, it is natural to desire that a node's neighbours are mapped to locations that lie close to the mapped location of the node itself. As before, let $\{\hat{\mathbf{x}}^{(i)}\}_{i=1}^N$ refer to the mapped location of all data points, and A_{ij} encode the (possibly weighted) adjacency of points $\mathbf{x}^{(i)}, \mathbf{x}^{(j)}$, i.e. $A_{ij} > 0$ if they are neighbours in the graph constructed from $\{\mathbf{x}^{(i)}\}_{i=1}^N$. Then we would like to find the mapped locations that minimise the cost function:

$$C := \frac{1}{2} \sum_{i,j=1}^{N} \|\hat{\mathbf{x}}^{(i)} - \hat{\mathbf{x}}^{(j)}\|_2^2 A_{ij},$$

subject to some constraint that keeps the locations scale-invariant. This cost function is very similar to the one we previously obtained for graph bipartitioning in Section 2. Indeed, it is a multivariate generalisation where the cost function can be written in terms of the combinatorial Laplacian L:

$$C = \mathrm{tr}(\widehat{X}^T L \widehat{X}),$$

where $\widehat{X}$ is the matrix obtained by stacking each data point's mapped location row-wise. It is no surprise then that the solution $\widehat{X}$ that minimises the cost function is given by the first m eigenvectors of a graph Laplacian, where the eigenvectors are arranged by non-decreasing eigenvalues — after ignoring the trivial eigenvector(s) corresponding to zero eigenvalue(s). If the constraint that renders the locations scale-invariant is given by (a) $\widehat{X}^T \widehat{X} = \mathbf{I}$ (where $\mathbf{I}$ is the identity matrix) then it corresponds to using eigenvectors of the combinatorial Laplacian L, whereas if it is given by (b) $\widehat{X}^T \mathcal{D} \widehat{X} = \mathbf{I}$ (where $\mathcal{D}$ is the degree matrix) then it corresponds to using eigenvectors of the normalised random-walk Laplacian L_{rw}. In other words, **the eigenvectors based on the Laplacian that are used to perform graph-based clustering can very well be used as a graph-based low-dimensional representation of the data**, and such a representation is referred to **spectral embedding** or **Laplacian eigenmaps**[3].

Extra: link between geodesic and geometry.
The notion of geodesic — or the shortest path — between points is intrinsically linked to the geometry of the underlying space (or manifold); it generalises the standard notion of a straight line in Euclidean space to non-Euclidean spaces. For instance, the "straight line" or geodesic between any two locations on the surface of the Earth — which is a sphere and thus exhibits spherical geometry — is given by the great circle that joins the two locations — a fact that is used to determine optimal flight paths. The behaviour of geodesics characterise the "shape" of the underlying space, and therefore the geodesic distances carry a lot of information about its intrinsic structure. In Euclidean space, for any straight line (geodesic) l and a point p not on l there is exactly one straight line l' through p that does not intersect l

[3]Belkin, M., & Niyogi, P. (2003). Laplacian Eigenmaps for Dimensionality Reduction and Data Representation. *Neural Computation, 15(6)*, 1373-1396 (see 'additional reading').

— this reflects Euclid's fifth postulate also termed as the parallel postulate. Hence, Euclidean space is said to be *flat* or with *zero curvature*. In spherical (or more generally elliptical) space, on the other hand, all "straight lines" intersect l, i.e. geodesics tend to converge and such a space is said to have a *positive* curvature. Finally, in hyperbolic space there are infinitely many such "straight lines" l' that do not intersect l, i.e. geodesics tend to diverge and such a space is said to have a *negative* curvature. For discrete objects like graphs, the "straight line" or geodesic between two nodes corresponds to a shortest path connecting them. The behaviour of geodesics on graphs can characterise their "curvature" and shape as well — for instance, a lattice-type graph is flat whereas a tree-like graph has negative curvature.

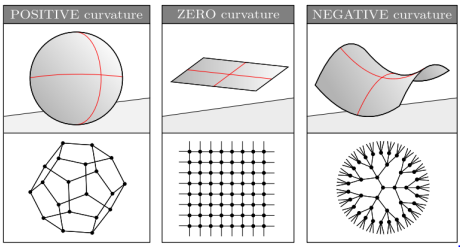


Figure from Devriendt, K., & Lambiotte, R. (2022). Discrete curvature on graphs from the effective resistance. *Journal of Physics: Complexity, 3(2)*, 025008.

3.2. Embeddings that preserve global structure. A potential problem with a method that preserve only local structure is that while it ensures that nodes that are adjacent (directly connected) on the graph are mapped close to one another in the low-dimensional space, it does not ensure that nodes that are far apart on the graph are mapped far away from one another. Consequently, if an embedding can preserve the (geodesic) distances or shortest path lengths between *all* pairs of nodes in the graph in the mapped space, then it preserves the global structure of the manifold.

In Chapter 10, although PCA was motivated as a dimensionality reduction method that minimises the reconstruction error, we saw that it ends up capturing the (maximum possible) variance in the data — with the sum of eigenvalues of the chosen top-m eigenvectors of the covariance matrix encoding the total variance explained (see the discussion after Figure 10.1). In other words, it tries to approximately preserve the Euclidean distance between every pair of points — thus preserving some global structure. Generalising beyond Euclidean distance, multidimensional scaling (MDS) is a class of methods that attempts to preserve pairwise distances, i.e., to minimise the cost function:

$$C := \sum_{i,j} \left[\|\hat{\mathbf{x}}^{(i)} - \hat{\mathbf{x}}^{(j)}\|_2^2 - d(i,j)^2 \right]^2,$$

where $d(i,j)$ refers to the distance between points i, j. In other words, when $d(i,j)$ is the Euclidean distance then we recover PCA. However, as we wish to preserve geodesic distances on the corresponding graph, we define $d(i,j)$ to be the geodesic distance or shortest path lengths in the graph. This algorithm is referred to as **Isomap**[4]. One can summarise the full algorithm as follows:

(1) Construct a (weighted) graph G (using techniques in Section 2 like k-nearest neighbours).

(2) Compute all pairwise geodesic distances in G (using Dijkstra's algorithm for graphs with positive-valued edge weights).

[4]Tenenbaum, J. B., Silva, V. D., & Langford, J. C. (2000). A Global Geometric Gramework for Nonlinear Dimensionality Reduction. *Science, 290(5500)*, 2319-2323 (see 'additional reading').

(3) Apply multidimensional scaling (MDS) to the matrix $D_{N\times N}$ of geodesic distances to obtain the mapping to a lower-dimensional space (equivalent to performing kernel PCA where the kernel matrix K is given by the (centered) negative element-wise squared distance matrix: $K_{ij} := -D_{ij}^2$)

Figure 11.12 demonstrates the Isomap algorithm applied to the Swiss roll dataset. Evidently, if the graph construction method trivially assumes a full-connected network ($\varepsilon \to \infty$ or $k \to \infty$) and weights the edges with the Euclidean distance in the original feature space, then Isomap is equivalent to PCA. On the other hand, assuming a fully-connected network and using a kernel matrix — such as one obtained from the radial basis function kernel — to provide edge weights is equivalent to kernel PCA. This shows that Isomap is a generalisation of kernel PCA, which in turn is a generalisation of PCA.

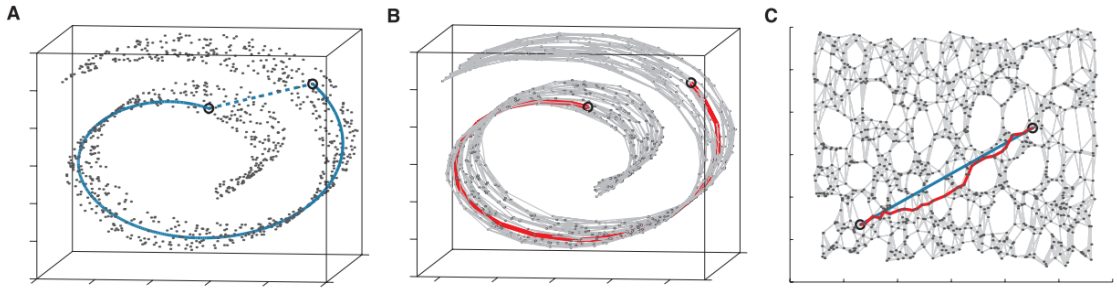


Figure 11.12. Using the Swiss roll dataset ($N = 1000$) to demonstrate the use of Isomap in performing nonlinear dimensionality reduction. Figure from Tenenbaum, Silva, & Langford (2000). (A) The Swiss roll dataset is not Euclidean as Euclidean distances do not capture the intrinsic non-linearity of the data. (B) Constructing a k-NN graph (with $k = 7$ neighbours) and using geodesic distances captures the non-linear structure well. (C) Performing MDS preserves pairwise distances on the graph (underlying 2D manifold) and "unrolls" the Swiss roll into two dimensions.

Quick Aside : As discussed before, the datasets can be transformed into a network of features as nodes and the edges represent a measure of linear or non-linear interdependency among the features, estimated across the N samples. Such a transformation provides an adjacency matrix of the dimension $p \times p$. Deploying the community detection algorithms discussed above on this graph will cluster groups of features that are strongly interdependent into say $c < p$ clusters. Now, if a coarse graining that combines the cluster of features into one (or more) components is applied on the original dataset, it will result in a dataset $\tilde{X}_{N\times c}$ with reduced dimensionality.

This transformation (also known as graph renormalisation) is used to reduce dimensionality in datasets with large number of features and each feature represent a different part of the system studied. Various coarse graining and renormalization algorithms have been proposed recently, see *Villegas, P., Gili, T., Caldarelli, G. et al. Laplacian renormalization group for heterogeneous networks. Nat. Phys. 19, 445–450 (2023)* and *Zhang, Z., Ghavasieh, A., Zhang, J. et al. Coarse-graining network flow through statistical physics and machine learning. Nat. Commun. 16, 1605 (2025).* This method of dimensionality reduction is a popular choice for time series datasets obtained from different regions of the brain, price of different stocks in the financial markets, population of different species in an ecosystem, spike trains obtained from a group of neurons etc. In all of these datasets, the data corresponding to each part of the system is the same in most cases (either electrical activity, price, population, spiking).

4. Graph centralities

In this very final section, we will introduce another concept from graph theory that can be very useful in understanding data from an alternative viewpoint.

The concept of *graph centrality* has to do with how to find the most important nodes or edges in a graph effectively establishing an importance ranking for nodes/edges. This notion is of course not uniquely defined, but rather a concept that can be viewed from many perspectives, and many notions of centrality are commonly used.

Recall that, according to our setup, the nodes/edges correspond to samples/relationships between samples in our data set. Hence when looking for the most important nodes, one is in essence making an assessment of the importance of samples, with the potential to rank samples or to find representative samples in the data set, and in turn, also outliers can be found thus. But of course, many different aims can be achieved through centrality measures.

The area of graph centralities is extremely well-developed in the literature and still the focus of open research, so in the following we give but a snippet, non-comprehensive list of some of the most commonly used centrality measures:

- **Degree centrality.** This is perhaps the simplest centrality. The importance of a node is simply defined to be its degree:

  $$\mathbf{c}_d = \frac{\mathbf{d}}{2E} = \frac{A\mathbf{1}}{2E}$$

  When applying this to a graph of citations for example, the importance of a single document is measured by the number of citations it received. In a social network, it would measure the number of contacts.

- **Betweenness centrality.** In this case we are not interested in searching for nodes that have a lot of connections, but rather nodes that have many paths going through them. Intuitively, the higher the betweenness, the more likely it is that removing the node would disconnect the graph or at least reduce the connectivity in the graph. In order to compute this measure, the set of shortest paths (i.e. the shortest path between every pair of nodes) in the whole graph needs to be found first. Then, the (normalised) number of minimal paths traversing each node is the betweenness centrality.

- **Closeness centrality.** This measure is based on distances in the graph: the closer a node is to all other nodes, the more central it is. For a given node, all shortest paths to all other nodes are computed and the closeness centrality is then defined to be the reciprocal of the average length of those paths:

  $$\mathbf{c}_c = \frac{1}{\frac{1}{N}\sum_j d_{ij}}$$

  Here we have defined the shortest path to be the measure of distance from one node to another.

- **Eigenvector centrality.** As opposed to the previous centralities, which are inspired by combinatorial graph-theoretical properties, we now look towards spectral graph properties once more. The first of such measures stems from the eigendecomposition of the adjacency matrix. Intuitively, this measure assigns higher centrality to those nodes that are themselves connected to other highly central nodes. What sounds like a circular argument, is simply based on the following statement: "The centrality of node i is a function of the centralities of

its neighbours", which can be written as

$$\mathbf{c}_e(i) = \alpha \sum_j A_{ij} \mathbf{c}_e(j),$$

Where α is a proportionality constant. This can be immediately identified as an eigenvalue/eigenvector problem:

$$A\mathbf{c}_e = \lambda \mathbf{c}_e$$

Hence the eigenvector centrality of each node is given by the component of the leading eigenvector of the adjacency matrix.

- **PageRank.** Arguably the most famous of them all, PageRank was introduced in the 90's at Stanford by Larry Page and Sergei Brin[5], who subsequently went on to found the company behind a search engine that is nowadays responsible for saving the lives of thousands of students every day. PageRank was of course originally conceived for ranking webpages on the internet, which is nothing other than one massive *directed* graph with websites for nodes and *directed* links for edges. Intuitively, PageRank will assign higher centrality to nodes where many searches will reside with higher probability, where the centrality of the nodes of those incoming edges also plays a role, i.e., being pointed at from an important node increases your score.

  PageRank is an iterative algorithm that is defined by a simple, yet powerful process (that is worth billions):

$$\mathbf{c}_{\mathrm{PR}\ t+1} = \alpha \left(A\mathcal{D}^{-1}\right) \mathbf{c}_{\mathrm{PR}\ t} + (1-\alpha)\frac{1}{N}\mathbf{1} \tag{11.9}$$

  For a given graph with adjacency matrix A, the PageRank centrality of each node is then given by the elements of the stationary eigenvector of (11.9), which is obtained by solving for $\mathbf{c}_{\mathrm{PR}}$ in:

$$\mathbf{c}_{\mathrm{PR}} = \alpha \left(A\mathcal{D}^{-1}\right) \mathbf{c}_{\mathrm{PR}} + (1-\alpha)\frac{1}{N}\mathbf{1}, \quad \alpha \in [0,1]$$

  The process (11.9) corresponds to a weighted *random walk* on the graph where the random walker follows the transition matrix of the graph $M = A\mathcal{D}^{-1}$ with probability α and transitions to any node in the graph with probability $(1-\alpha)$. Note that in this case the graph is directed and $A \neq A^T$. The parameter α, called the teleportation parameter, regulates how much the random walk follows the graph or jumps unrestricted by it. This parameter is needed to guarantee ergodicity of the random walk and is customarily set to $\alpha = 0.85$. This model of the random walk was inspired by web surfing.

  What that big company is doing now, only they know, but it all started with this formula and this simple idea based on random walks on graphs.

[5] Page, L., Brin, S., Motwani, R. and Winograd, T. (1999) "The PageRank Citation Ranking: Bringing Order to the Web". Technical Report, Stanford InfoLab (see 'additional reading').